

MODELO DE DATOS - SUPABASE

Tablas, propiedades, claves y relaciones propuestas

Fuente funcional: TP 1 - Programación IV - 2026 C2, intercambio de emails incluido en el PDF entregado.

1. Criterio de modelado

La propuesta usa PostgreSQL, que es la base de Supabase, con claves UUID y relaciones mediante foreign keys. Se separa autenticación de la información de negocio: `auth.users` queda gestionada por Supabase y `public.perfiles` almacena los datos del cliente. Las tablas de negocio se normalizan para evitar duplicación, soportar reportes y mantener integridad referencial.

Advertencia de alcance

Lo que sigue es una propuesta de diseño de datos derivada de los requisitos. El intercambio de mails no define nombres de tablas, tipos exactos ni restricciones SQL; por eso esos elementos son decisiones de modelado y no requisitos textuales del cliente.

2. Diagrama conceptual

Diagrama simplificado por bloques funcionales. Las cardinalidades exactas están en la sección 4

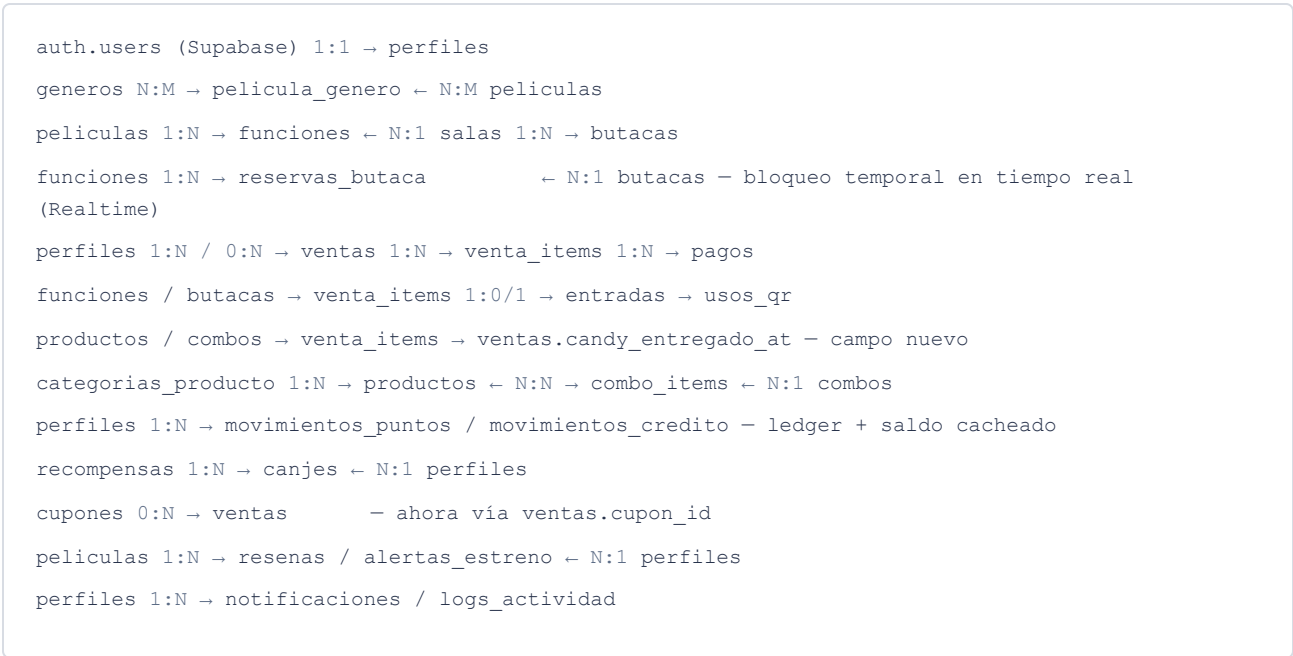


Figura 1 — Relación conceptual propuesta (revisada). Algunas relaciones operativas se resuelven a través de claves y consultas, no como relaciones directas entre todas las tablas.

3. Tablas principales y propiedades

perfiles

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK; referencia a auth.users.id	Identidad del usuario registrado
nombre	text	NOT NULL	Nombre
apellido	text	NOT NULL	Apellido
fecha_nacimiento	date	NOT NULL	Fecha para cálculo de edad
tipo_sangre	text	NULL	Dato solicitado
color_ojos	text	NULL	Dato solicitado
dias_vacaciones_anuales	integer	NULL	Dato solicitado
rol	text	NOT NULL; CHECK; DEFAULT 'cliente'	cliente / empleado / admin
credito_saldo	numeric(12,2)	NOT NULL DEFAULT 0	Crédito disponible (cacheado desde movimientos_credito)
puntos_saldo	integer	NOT NULL DEFAULT 0	Puntos disponibles (cacheado desde movimientos_puntos, misma lógica que credito_saldo)
created_at	timestamptz	NOT NULL	Alta
updated_at	timestamptz	NOT NULL	Última modificación

Cambio: se agrega puntos_saldo para que el saldo de puntos se lea igual de rápido que el crédito, en vez de sumar el ledger completo cada vez que el usuario abre su perfil. Ambos saldos se mantienen sincronizados por trigger sobre sus respectivas tablas de movimientos. En la Fase 0.6, rol se fijó con DEFAULT 'cliente' (todo alta pública arranca como cliente; admin/empleado se asignan a mano) y se agregó un trigger que impide que el propio usuario modifique rol, credito_saldo o puntos_saldo desde el cliente.

peliculas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
nombre	text	NOT NULL	Título
duracion_minutos	integer	NOT NULL	Duración usada para agenda
imagen_url	text	NOT NULL	Póster/imagen
sinopsis	text	NOT NULL	Descripción
clasificacion_edad	integer / null	CHECK 13/18/null	Restricción
fecha_estreno	date	NOT NULL	Estreno
publicada	boolean	NOT NULL DEFAULT true	Visible en catálogo
preventa_habilitada	boolean	NOT NULL DEFAULT false	Permite preventa

precio_preventa	numeric(12,2)	NULL; CHECK > 0	Precio; obligatorio con preventa
created_at	timestampz	NOT NULL	Alta

Nota: la ventana de preventa ("7 días antes del estreno, vuelve al precio normal después") se calcula en la aplicación a partir de fecha_estreno y no necesita columna propia; el booleano preventa_habilitada es el interruptor manual del admin para abrir/cerrar antes de tiempo si hiciera falta.

generos

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
nombre	text	UNIQUE lower(nombre); NOT NULL	Nombre del género

pelicula_genero

Propiedad	Tipo	Clave/restricción	Uso
pelicula_id	uuid	PK/FK	Película
genero_id	uuid	PK/FK	Género

salas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
nombre	text	UNIQUE; NOT NULL	Nombre/número de sala
activa	boolean	NOT NULL DEFAULT true	Habilitación

butacas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
sala_id	uuid	FK -> salas	Sala
fila	text	NOT NULL	A-T
numero	integer	NOT NULL	Número
tipo	text	CHECK normal / accesible / vip	Diferencia por tipo
precio_adicional	numeric(12,2)	NOT NULL DEFAULT 0	Diferencia por tipo
activa	boolean	NOT NULL DEFAULT true	Habilitación
UNIQUE	-	(sala_id, fila, numero)	Evita duplicados

La tabla es genérica a propósito (no fija filas/columnas en el esquema): el layout real del cliente — 20 filas, columnas de 4/20/4 butacas, filas J/K convertidas en butacas accesibles (2/10/2), VIP en filas R/S/T — se carga como datos de seed (Fase 0.7), no como estructura. Así una sala nueva con otra distribución no requiere cambiar el esquema.

reservas_butaca

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
funcion_id	uuid	FK -> funciones	Función para la que se reserva
butaca_id	uuid	FK -> butacas	Butaca bloqueada temporalmente
sesion_id	text	NOT NULL	Carrito/sesión (usuario o anónimo) que la seleccionó
expira_at	timestampz	NOT NULL	Vencimiento del bloqueo (propuesto: 10 minutos)
created_at	timestampz	NOT NULL	Alta
UNIQUE	-	(funcion_id, butaca_id) — ver regla 5.1	Evita reservar la misma butaca dos veces mientras el bloqueo está vigente

Cubre un requisito explícito del cliente: "cuando un usuario está seleccionando butacas, debe ver cuáles ya están ocupadas por otra compra en ese mismo momento". Sin esta tabla, el mapa de butacas solo reflejaba ventas ya confirmadas, no selecciones en curso. Se escucha por Supabase Realtime y se limpia por expiración (ver regla 5.1). En la Fase 0.6 la tabla se creó con RLS de lectura pública (para el mapa en tiempo real) y sin políticas de escritura para el cliente: el bloqueo/liberación lo hace el frontend con políticas RLS propias, tal como pauta la regla 5.1.

funciones

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
pelicula_id	uuid	FK	Película
sala_id	uuid	FK	Sala asignada automáticamente
inicio	timestampz	NOT NULL	Inicio
fin	timestampz	NOT NULL	Calculado por duración
tipo_proyeccion	text	CHECK 2D/3D/4D/5D	Tecnología
idioma	text	CHECK castellano / subtitulada	Idioma
precio_base	numeric(12,2)	NOT NULL	Precio base
estado	text	CHECK programada/cancelada/finalizada	Estado
created_by	uuid	FK -> perfiles	Administrador que creó
created_at	timestampz	NOT NULL	Fecha de creación

El bloqueo de solapamientos (margen de 30 minutos) es un EXCLUDE constraint sobre (sala_id, tstzrange(inicio, fin + 30 min)); ver regla 5.2 con el SQL exacto. La asignación automática de sala la resuelve la aplicación al guardar la función: busca las salas activas sin funciones en ese rango y elige la primera libre (o valida la sala elegida a mano); el EXCLUDE queda como respaldo. La aplicación envía fin ya calculado (inicio + duración de la película); el trigger de la Fase 0.6 solo lo completa si llega vacío. created_by se completa con el administrador que da de alta la función.

ventas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK NULL	Cliente; null si anónimo
email_contacto	text	NOT NULL	Contacto para compra anónima
fecha_nacimiento_comprador	date	NULL; obligatorio si usuario_id es NULL y la función es de una película con clasificacion_edad no nula	Valida la restricción de edad en compra anónima
cupon_id	uuid	FK NULL -> cupones	Cupón aplicado a la compra, si hubo uno
subtotal	numeric(12,2)	NOT NULL	Importe previo
descuento	numeric(12,2)	NOT NULL DEFAULT 0	Descuento aplicado
credito_usado	numeric(12,2)	NOT NULL DEFAULT 0	Crédito aplicado
puntos_usados	integer	NOT NULL DEFAULT 0	Puntos canjeados
total	numeric(12,2)	NOT NULL	Total
estado	text	CHECK pendiente/pagada/cancelada	Estado
candy_entregado_at	timestampz	NULL	Momento de entrega del candy de la compra, independiente de la validación de las entradas
created_at	timestampz	NOT NULL	Fecha de compra
cancelled_at	timestampz	NULL	Fecha cancelación

fecha_nacimiento_comprador: resuelve que la compra anónima permitida por el cliente ("pueden comprar siendo anónimos, siempre que paguen no hay problema") choca con la restricción de edad en películas +13/+18. Decisión tomada: si no hay usuario logueado, el checkout pide la fecha de nacimiento igual que a un usuario registrado, y se valida contra clasificacion_edad de la misma forma.

candy_entregado_at: el mail dice que "con el mismo QR puedan retirar" el candy, y que el QR "deja de funcionar" cuando se valida la entrada o se entrega la comida — son dos eventos independientes. Se modela a nivel de venta (no por línea de producto) asumiendo que el candy de una compra se retira en un solo evento con el QR de cualquiera de sus entradas. A confirmar con el cliente si en cambio quiere entregas parciales por producto.

cupon_id : el diagrama conceptual (sección 2) y el mapeo de módulos (sección 6) ya daban por sentado un vínculo cupones→ventas que nunca se había formalizado como columna. Se agrega nullable porque no toda venta usa cupón; queda separado de descuento (que sigue siendo el importe efectivamente descontado, sea por cupón, crédito o combinación) para poder reportar qué cupón específico se usó (M12) y, a futuro, controlar cupones de un solo uso por comprador.

venta_items

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
venta_id	uuid	FK	Compra
tipo_item	text	CHECK entrada/producto/combo/recompensa	Tipo de línea
producto_id	uuid	FK NULL	Producto
combo_id	uuid	FK NULL	Combo
funcion_id	uuid	FK NULL	Función si es entrada
butaca_id	uuid	FK NULL	Butaca si es entrada
cantidad	integer	NOT NULL	Cantidad
precio_unitario	numeric(12,2)	NOT NULL	Precio al momento de compra
total_linea	numeric(12,2)	NOT NULL	Subtotal de línea
cancelado	boolean	NOT NULL DEFAULT false	Se sincroniza por trigger con ventas.estado='cancelada'
CHECK	-	coherencia por tipo_item — ver regla 5.3	Evita filas mixtas (ej. tipo_item='entrada' con producto_id seteado)
UNIQUE parcial	-	(funcion_id, butaca_id) WHERE tipo_item='entrada' AND cancelado=false — ver regla 5.3	Impide vender la misma butaca dos veces para la misma función

Antes, la disponibilidad de butacas dependía solo de la validación al confirmar la compra. Se agrega cancelado desnormalizado (en vez de hacer join contra ventas.estado) para poder sostener el índice único parcial, que es la defensa de integridad de fondo; la validación del frontend sigue siendo la primera línea de validación, esto es la segunda.

entradas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
venta_item_id	uuid	FK UNIQUE	Item de entrada
codigo_qr	text	UNIQUE; NOT NULL	Código de validación (también ingresable a mano)
estado	text	CHECK emitida/validada/cancelada	Estado del ingreso a sala
validada_at	timestampz	NULL	Momento de acceso a la sala
adulto_requerido	boolean	NOT NULL DEFAULT false	Aviso de edad

Esta tabla no cambia de estructura: sigue representando solo el ingreso a la sala. La entrega de candy de la compra se trackea aparte en ventas.candy_entregado_at, precisamente porque son dos redenciones independientes del mismo QR.

categorias_producto

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
nombre	text	UNIQUE; NOT NULL	Categoría

productos

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
categoria_id	uuid	FK	Categoría
nombre	text	NOT NULL	Producto
descripcion	text	NULL	Descripción
precio	numeric(12,2)	NOT NULL	Precio
stock	integer	NULL	Stock si se desea controlar
activo	boolean	NOT NULL DEFAULT true	Disponible

combos

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
nombre	text	NOT NULL	Nombre
descripcion	text	NULL	Descripción
precio_fijo	numeric(12,2)	NOT NULL	Precio
destacado	boolean	DEFAULT false	Mostrar destacado

activo	boolean	DEFAULT true	Disponible
--------	---------	--------------	------------

combo_items

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
combo_id	uuid	FK	Combo
producto_id	uuid	FK	Producto
cantidad	integer	NOT NULL	Cantidad incluida

cupones

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
codigo	text	UNIQUE	Código
porcentaje	numeric(5,2)	NOT NULL	Descuento
tipo	text	CHECK primera_compra / edad / general	Tipo
edad_minima	integer	NULL; requerido si tipo='edad'	Por ejemplo 50
activo	boolean	DEFAULT true	Vigente
fecha_inicio	timestamptz	NULL	Vigencia
fecha_fin	timestamptz	NULL	Vigencia

Referenciada desde `ventas.cupon_id` — ver página 8.

recompensas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
tipo	text	CHECK entrada/producto	Tipo
producto_id	uuid	FK NULL	Producto si corresponde
nombre	text	NOT NULL	Descripción
puntos_costo	integer	NOT NULL	Costo en puntos
activo	boolean	DEFAULT true	Disponible

movimientos_puntos

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK	Cliente
venta_id	uuid	FK NULL	Compra origen
canje_id	uuid	FK NULL	Canje origen
tipo	text	CHECK acreditacion/debito/ajuste	Tipo
puntos	integer	NOT NULL	Cantidad positiva/negativa según criterio
created_at	timestamptz	NOT NULL	Fecha

1 punto por cada peso gastado, calculado sobre la parte de `ventas.total` efectivamente pagada (excluyendo `credito_usado` y el valor cubierto por `puntos_usados`), para no generar puntos sobre plata que no entró.

canjes

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK	Cliente
recompensa_id	uuid	FK	Recompensa
venta_id	uuid	FK NULL	Compra donde se aplicó
puntos_usados	integer	NOT NULL	Costo
created_at	timestampz	NOT NULL	Fecha

movimientos_credito

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK	Cliente
venta_id	uuid	FK NULL	Compra origen
tipo	text	CHECK acreditacion/uso/ajuste	Tipo
monto	numeric(12,2)	NOT NULL	Monto
created_at	timestampz	NOT NULL	Fecha

Misma convención que movimientos_puntos.puntos: monto positivo o negativo según el tipo de movimiento.

resenas

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
pelicula_id	uuid	FK	Película
usuario_id	uuid	FK	Cliente
estrellas	smallint	CHECK 1-5	Calificación
comentario	text	CHECK longitud corta	Comentario
created_at	timestamptz	NOT NULL	Fecha
UNIQUE	-	(pelicula_id, usuario_id)	Una reseña por usuario/película

alertas_estreno

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK	Cliente
pelicula_id	uuid	FK	Película
activa	boolean	DEFAULT true	Alerta activa
created_at	timestamptz	NOT NULL	Alta
UNIQUE	-	(usuario_id, pelicula_id)	Evita duplicados

notificaciones

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK	Cliente
tipo	text	NOT NULL	Tipo de aviso
titulo	text	NOT NULL	Título
mensaje	text	NOT NULL	Contenido
leida	boolean	DEFAULT false	Estado
created_at	timestamptz	NOT NULL	Fecha

pagos

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
venta_id	uuid	FK	Compra
metodo	text	NOT NULL	Proveedor/método
monto	numeric(12,2)	NOT NULL	Monto
estado	text	CHECK pendiente/aprobado/rechazado	Estado
referencia_externa	text	NULL	ID de operación si existe
created_at	timestampz	NOT NULL	Fecha

usos_qr

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
codigo_qr	text	NOT NULL	Código validado (escaneado o ingresado a mano)
entrada_id	uuid	FK NULL -> entradas	Traza la entrada asociada al código, cuando existe match
tipo	text	CHECK entrada/candy	Qué se está validando
empleado_id	uuid	FK -> perfiles	Empleado
resultado	text	CHECK validado/rechazado	Resultado
created_at	timestampz	NOT NULL	Fecha y hora

entrada_id nullable porque un intento con código inexistente o ya vencido igual debe quedar logueado (resultado='rechazado') sin poder resolver a una entrada real.

logs_actividad

Propiedad	Tipo	Clave/restricción	Uso
id	uuid	PK	Identificador
usuario_id	uuid	FK -> perfiles	Quién ejecutó
accion	text	NOT NULL	Qué hizo
entidad	text	NOT NULL	Sobre qué entidad
entidad_id	uuid	NULL	Registro afectado
detalle	jsonb	NULL	Valores relevantes
created_at	timestampz	NOT NULL	Fecha y hora

4. Relaciones principales

Origen	Destino	Cardinalidad	Interpretación
auth.users	perfiles	1:1	Un usuario autenticado tiene un perfil de negocio.
perfiles	ventas	1:N / 0:N	Una venta puede pertenecer a un cliente o ser anónima.
cupones	ventas	1:N / 0:N	Una venta puede aplicar un cupón, o ninguno, vía ventas.cupon_id.
peliculas	generos	N:M	Mediante pelicula_genero.
salas	butacas	1:N	Una sala posee sus butacas.
peliculas	funciones	1:N	Una película se proyecta en múltiples funciones.
salas	funciones	1:N	Una sala aloja funciones a lo largo del tiempo; debe impedirse solapamiento.
funciones	reservas_butaca	1:N	Bloqueos temporales de butacas en selección, expiran por TTL.
butacas	reservas_butaca	1:N	Una butaca puede tener bloqueos sucesivos a lo largo del tiempo (nunca vigentes en simultáneo).
funciones	venta_items	1:N	Los items de entrada apuntan a una función.
ventas	venta_items	1:N	Una compra contiene múltiples items.
venta_items	entradas	1:0/1	Cada item de entrada genera un ticket/QR.
productos	venta_items	1:N	Un producto puede aparecer en muchas ventas.
combos	combo_items	1:N	Un combo está compuesto por productos.
productos	combo_items	1:N	Un producto puede participar en varios combos.
perfiles	movimientos_puntos	1:N	Ledger de puntos del cliente.
perfiles	movimientos_credito	1:N	Ledger de crédito del cliente.
entradas	usos_qr	1:N	Cada intento de escaneo (válido o no) queda trazado; entrada_id puede repetirse por escaneo fallido reintentado.
peliculas	resenas	1:N	Una película recibe reseñas de múltiples usuarios.
perfiles	alertas_estreno	1:N	Un cliente puede seguir múltiples estrenos.
perfiles	notificaciones	1:N	Un cliente recibe múltiples avisos.
perfiles	logs_actividad	1:N	Un usuario interno genera eventos auditables.

5. Reglas SQL / integridad recomendadas

Usar foreign keys y restricciones NOT NULL / CHECK / UNIQUE para proteger integridad.

5.1 — reservas_butaca: bloqueo temporal en selección

No alcanza un índice único simple porque la vigencia depende de `expira_at` (una condición que cambia con `now()`), no evaluable en un índice parcial clásico). Se resuelve con upsert transaccional:

```
-- Upsert de la reserva, con la fila bajo lock:
INSERT INTO reservas_butaca (funcion_id, butaca_id, sesion_id, expira_at)
SELECT :funcion_id, :butaca_id, :sesion_id, now() + interval '10 minutes'
WHERE NOT EXISTS (
  SELECT 1 FROM reservas_butaca
  WHERE funcion_id = :funcion_id AND butaca_id = :butaca_id
    AND expira_at > now()
  FOR UPDATE
);
-- Limpieza de vencidas: lazy (al consultar disponibilidad) o cron/Edge Function periódico.
```

5.2 — funciones: no solapar horarios en la misma sala

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE funciones ADD CONSTRAINT sin_solapamiento_por_sala
EXCLUDE USING gist (
  sala_id WITH =,
  tstzrange(inicio, fin + interval '30 minutes') WITH &&
);
```

5.3 — venta_items: coherencia por tipo_item y no vender la misma butaca dos veces

```
ALTER TABLE venta_items ADD CONSTRAINT chk_coherencia_tipo_item CHECK (  
  (tipo_item = 'entrada'      AND funcion_id IS NOT NULL AND butaca_id IS NOT NULL  
                                AND producto_id IS NULL AND combo_id IS NULL) OR  
  (tipo_item = 'producto'    AND producto_id IS NOT NULL  
                                AND combo_id IS NULL AND funcion_id IS NULL AND butaca_id IS NULL) OR  
  (tipo_item = 'combo'       AND combo_id IS NOT NULL  
                                AND producto_id IS NULL AND funcion_id IS NULL AND butaca_id IS NULL) OR  
  (tipo_item = 'recompensa')  
);  
  
CREATE UNIQUE INDEX ux_butaca_por_funcion ON venta_items (funcion_id, butaca_id)  
  WHERE tipo_item = 'entrada' AND butaca_id IS NOT NULL AND cancelado = false;  
  
-- Trigger sobre ventas: al pasar estado a 'cancelada', propaga cancelado=true  
-- a sus venta_items para liberar el índice único de arriba.
```

Para butacas vendidas, validar disponibilidad en backend/transacción y no confiar solo en el estado visual del frontend — el índice único de 5.3 es la red de contención, no el mecanismo principal.

El saldo de puntos y crédito puede derivarse de movimientos (ledger) para conservar trazabilidad; perfiles.credito_saldo y perfiles.puntos_saldo son cachés mantenidos por trigger sobre movimientos_credito / movimientos_puntos respectivamente.

Las operaciones sensibles de QR deben ser atómicas: buscar, verificar estado y consumir en una misma operación lógica (un único update condicionado al estado) para evitar doble validación concurrente; aplica a entradas.estado y a ventas.candy_entregado_at.

Los logs deben escribirse desde backend/triggers/servicios para que no dependan de que un cliente decida registrar su propia actividad.

Las políticas RLS de Supabase deben limitar cada registro a su usuario y reservar las operaciones administrativas a perfiles con rol apropiado. reservas_butaca necesita una política que permita lectura pública (para pintar el mapa en tiempo real) pero escritura solo del propio dueño de la sesión.

Implementado en la Fase 0.6

Todo lo anterior de esta sección ya está aplicado en `supabase/migrations/: 5.1` y la escritura de `reservas_butaca` quedan para el frontend con RLS (Fase 2); 5.2 y 5.3 son constraints activos desde esta fase. RLS quedó habilitada en las 26 tablas de negocio con el criterio descrito en el propio archivo de políticas del repo.

6. Relación entre el modelo y los módulos funcionales

Módulo	Tablas
M01 Usuarios	perfiles, auth.users
M02 Catálogo	peliculas, generos, pelicula_genero, resenas
M03 Salas/Butacas	salas, butacas, reservas_butaca
M04 Funciones	funciones
M05 Venta	ventas (incl. fecha_nacimiento_comprador, cupon_id), venta_items, pagos
M06 Candy/Combos	categorias_producto, productos, combos, combo_items
M07 Promos	cupones, peliculas, ventas.cupon_id
M08 Fidelización	movimientos_puntos, recompensas, canjes, movimientos_credito, perfiles.puntos_saldo
M09 Tickets/QR	entradas, usos_qr, ventas.candy_entregado_at
M10 Cancelación	ventas, movimientos_credito, entradas
M11 Alertas	alertas_estreno, notificaciones
M12 Reportes	consultas/vistas sobre ventas, entradas, funciones, productos
M13 Auditoría	logs_actividad

7. Decisiones y puntos a confirmar

- La entidad auth.users es de Supabase y no debería duplicarse en una tabla propia; public.perfiles guarda los atributos de negocio requeridos.
- El rol puede mantenerse en perfiles, aunque una implementación más robusta puede separar roles/permisos en tablas propias si el proyecto crece.
- **Resuelto (2026-09-16):** la compra anónima en películas con restricción de edad exige capturar fecha_nacimiento_comprador en el checkout, validado igual que con un perfil registrado.
- **Resuelto (2026-09-16):** el vínculo cupones→ventas, mencionado en el diagrama conceptual pero nunca formalizado, se resuelve agregando ventas.cupon_id (nullable) al escribir el esquema SQL de la Fase 0.6.
- A confirmar: candy_entregado_at se modela a nivel de venta completa. Si el cliente pide entregar el candy en partes (por producto/línea), habría que bajar el campo a venta_items en su lugar.
- A confirmar: el TTL de reservas_butaca se propone en 10 minutos; ajustar según lo que se decida para la experiencia de checkout.
- No se modela un mapa interno del edificio porque el cliente dijo que todavía no estaba aprobado.
- La canalización exacta de notificaciones y del proveedor de pagos debe definirse antes de cerrar la capa de infraestructura.
- **Resuelto (2026-10-05):** el checkout permite que un cliente registrado use puntos como pago parcial, a razón de 1 punto = \$1 (simétrico con RN08) y combinables con crédito. ventas.puntos_usados y credito_usado reducen lo que se cobra; el saldo disponible se valida en el checkout.

- **Resuelto (2026-10-05):** el proveedor de pagos es simulado. La compra se graba desde el frontend: la venta se inserta en pendiente y pasa a pagada al final, y se registra una fila de pagos por cada medio con monto mayor a cero (metodo credito, puntos o tarjeta, todas aprobado); la de tarjeta guarda en referencia_externa el código de autorización simulado. Los puntos se acreditan sobre el monto de tarjeta. entradas.adulto_requerido se marca cuando la película tiene clasificación de edad.
- **Resuelto (2026-10-07):** los canjes de recompensas se hacen dentro de una compra (CU-13), no como un paso previo: el cliente registrado elige las recompensas en el checkout y cada canje se graba con su venta_id y un débito en movimientos_puntos con su canje_id. Lo canjeado vale 0 en la venta: la entrada cubierta queda como venta_items tipo entrada (la de menor precio de la compra) y el producto como tipo recompensa con su producto_id, que descuenta stock. Los puntos de recompensas y los usados como pago parcial salen del mismo saldo.
- **Resuelto (2026-10-07):** cancelación (RN07, CU-22/CU-27). Se cancela la venta completa pasando ventas.estado de pagada a cancelada; el cliente puede hacerlo hasta 2 horas antes de la función y el administrador en cualquier momento, con un motivo obligatorio que se guarda en la nueva columna ventas.motivo_cancelacion (text, NULL; queda nulo cuando cancela el cliente). El crédito acreditado en movimientos_credito es total – puntos_usados, es decir lo pagado con tarjeta o con crédito; los puntos no vuelven como crédito: se revierte el neto de movimientos_puntos de la venta (los usados como pago o en canjes vuelven al saldo y se descuentan los acreditados por la compra) con un movimiento tipo ajuste. Una compra anónima cancelada libera las butacas sin generar crédito. Todo lo hace el trigger de cancelación sobre ventas (que ya propagaba venta_items.cancelado), junto con entradas.estado = cancelada; así el administrador no necesita escribir en los ledgers de otro usuario. No se repone stock del Candy Bar.
- **Resuelto (2026-10-07):** preventa y avisos de venta (RN06, CU-28/CU-29). La ventana de preventa se calcula en la aplicación a partir de fecha_estreno: con preventa_habilitada la venta abre 7 días antes del estreno y, hasta ese día, cada entrada cuesta precio_preventa (más el precio_adicional de la butaca) en lugar de funciones.precio_base; sin preventa, la venta abre el día del estreno. Mientras tanto la película solo figura en Próximamente, donde el cliente activa su alerta. Las notificaciones son in-app (ni push ni email): al abrir la aplicación con sesión, el frontend busca las alertas activas cuya película ya está a la venta, inserta la notificación del cliente y pasa alertas_estreno.activa a false (alerta cumplida). Para eso se agregó la columna notificaciones.pelicula_id (uuid, NULL, FK a peliculas) que enlaza el aviso con la película, y una política que deja al cliente insertar solo notificaciones propias y no leídas; el contenido sigue sin poder modificarse después.
- **Resuelto (2026-10-07):** validación presencial (RN05, CU-24/CU-25). candy_entregado_at queda a nivel de venta completa, como se proponía arriba: el Candy de una compra se retira en un solo evento con el QR de cualquiera de sus entradas, siempre que la venta esté pagada e incluya productos, combos o recompensas de producto. El empleado (o el administrador) busca la entrada por codigo_qr, escaneado con la cámara o ingresado a mano, y la consume con un update condicionado al estado: entradas.estado pasa de emitida a validada junto con validada_at, y ventas.candy_entregado_at solo se completa si seguía en NULL; si otro puesto ya lo consumió, el update no afecta filas y el intento se rechaza. Cada intento, aceptado o rechazado, se registra desde el frontend en usos_qr con el empleado_id de la sesión (entrada_id nulo si el código no existe). Nuevas políticas RLS permiten esos updates y el insert en usos_qr solo a los roles empleado y admin, y dejan que cada empleado lea sus propios usos_qr para ver el historial de su sesión. Un trigger sobre ventas solo deja completar candy_entregado_at al personal y no permite cambiarlo una vez completo.

Fin del documento